
STEP 1: 1-BIT OPERATIONS

BLUE GIRAFFES

Connor Nauert, Daniel Okoh, James Webb

CS.3339 Computer Architecture

Texas State University

September 17, 2026

1 Introduction

In this project, we are designing a small computer from scratch using Verilog for hardware programming. We will also be using Gtkwave to analyze test case waveforms and display the inputs and outputs of the gates.

Step 1 involves constructing the initial simple 1-bit gates.

2 Verilog Code

In this section, we will go over every circuit and the associated Verilog code for each one, including the test modules.

2.1 Not Circuit

The Not circuit is the most simple of the 1-bit gates. One input, A, is inverted to an output X. If A is a 1, X will be a 0. If A is 0, X will be a 1.

```
1 module not_test (A, X);
2
3     input A;
4     output X;
5
6     assign X = !A;
7
8 endmodule
```

Testing the Not circuit involves creating a register A and measuring an output X. There are two possible states to be tested: when A is 0 and when A is 1. The pound symbol in the snippet below calls for the computer to wait for some amount of seconds, which allows us to observe how the gate acts over time.

```
1 `timescale 1ns / 1ns
2 `include "not_test.v"
3
4 module not_tb;
5
6     reg A;
7     wire X;
8
9     not_test uut(A, X);
10
11     initial begin
```

```

12
13 $dumpfile("not_tb.vcd");
14 $dumpvars(0, not_tb);
15
16 A = 0;
17 #20;
18
19 A = 1;
20 #20;
21
22 $display("Testing not");
23
24 end
25
26 endmodule

```

2.2 Nand Circuit

The Nand circuit is composed of a Not gate and an And gate. Two inputs, A and B, are inputted into an And gate. The output of the And gate is then inverted and outputted to X. This means that if A and B are both 1, the And gate will invert the combined 1 into a 0. The other three states will output a 1.

```

1 module nand_test (A, B, X);
2
3     input A, B;
4     output X;
5
6     assign X = !(A && B);
7
8 endmodule

```

To test the Nand circuit, we have created two registers, A and B, as well as a wire X. This way we are able to take two inputs at a time and test each possible input for the circuit. If it's working correctly, X should be equal to 1 for all inputs except when both A and B are equal to 1.

```

1 `timescale 1ns / 1ns
2 `include "nand_test.v"
3
4 module nand_tb;
5
6 reg A, B;

```

```
7 wire X;
8
9 nand_test uut(A, B, X);
10
11 initial begin
12
13     $dumpfile("nand_tb.vcd");//holds output waveform
14     $dumpvars(0, nand_tb);
15
16     A = 0;
17     B = 0;
18     #10;
19
20     A = 0;
21     B = 1;
22     #10;
23
24     A = 1;
25     B = 0;
26     #10;
27
28     A = 1;
29     B = 1;
30     #10;
31
32     $display("Testing nand");
33
34 end
35
36 endmodule
```

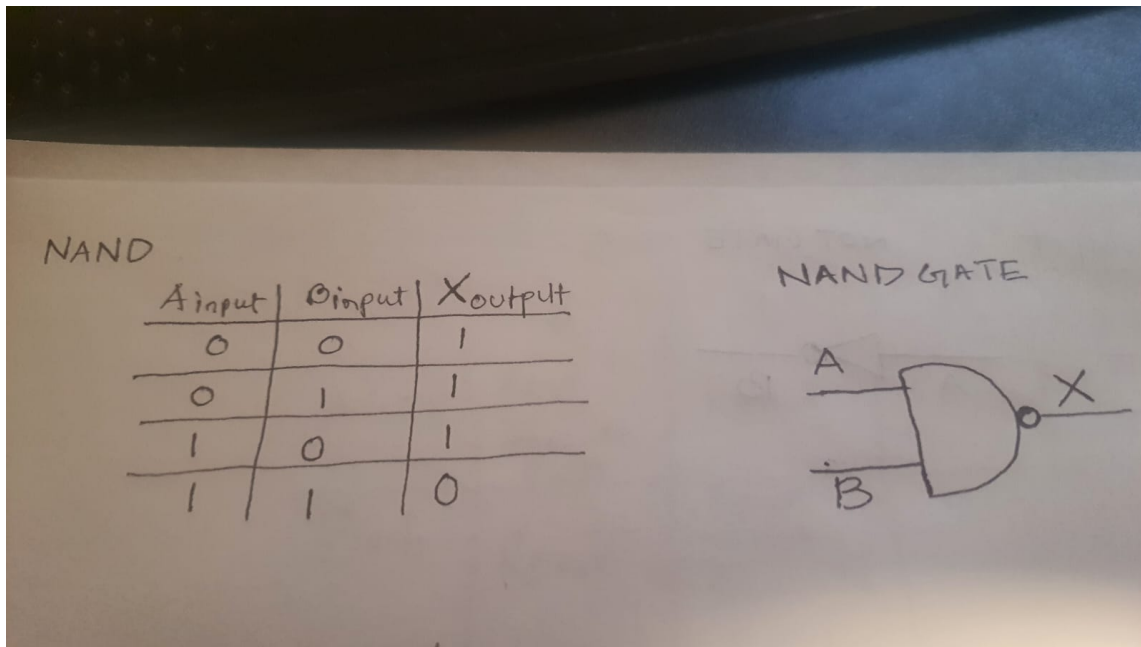


Figure 1: Nand truth table and gate

2.3 Nor Circuit

The Nor circuit is similar to the Nand circuit in that it is the inverse of another gate. The Or gate will take two inputs, A and B. The Not gate will switch the bit outputted by the Or gate, which is then wired to X. If both A and B are 0, then X should be 1. Otherwise, X is 0.

```

1 // all I/O in args
2 // first try and and then try nor
3
4 module nor_test (A, B, X);
5
6     input A, B;
7     output X;
8
9     assign X = !(A || B);
10
11 endmodule

```

The Nor circuit testbench also uses two inputs, one output, and four possible states.

```

1 `timescale 1ns / 1ns
2 `include "nor_test.v"
3
4 module nor_tb;

```

```
5
6 reg A, B;
7 wire X;
8
9 nor_test uut(A, B, X);
10
11 initial begin
12
13     $dumpfile("nor_tb.vcd");//holds output waveform
14     $dumpvars(0, nor_tb);
15
16     A = 0;
17     B = 0;
18     #10;
19
20     A = 0;
21     B = 1;
22     #10;
23
24     A = 1;
25     B = 0;
26     #10;
27
28     A = 1;
29     B = 1;
30     #10;
31
32     $display("Testing nor");
33
34 end
35
36 endmodule
```

2.4 1x4 Arithmetic Shift Circuit

3 Waveform Tests

In this section we will showcase the waveforms created using our testbenches for each circuit we coded in Verilog. We used GTKWave to create these waveforms.

3.1 Nand Circuit Waveform

At 0 ns, we can see that both A and B are 0, so the output X is 1.

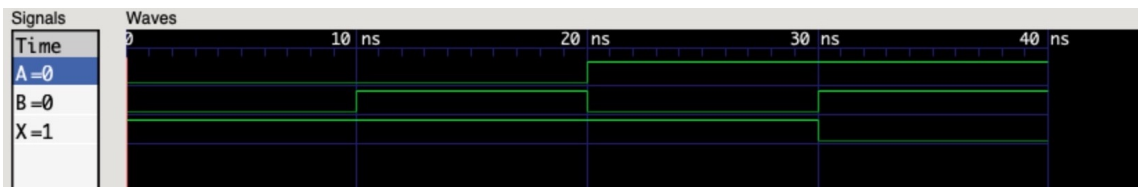


Figure 2: Nand Circuit with marker at 0ns

At 20 ns, A is 1 and B is 0, so X still remains at 1.

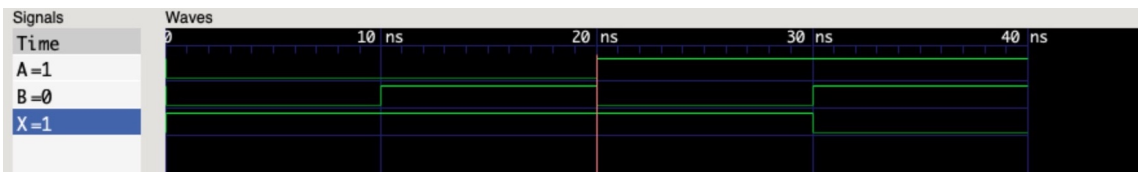


Figure 3: Nand Circuit with marker at 20ns

At 40 ns, both A and B are 1, so X is now 0 because the Nand condition has been broken.

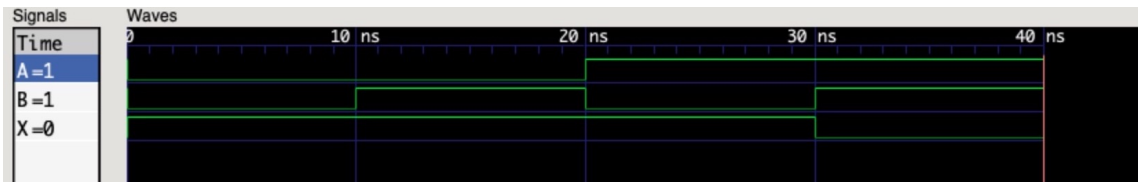


Figure 4: Nand Circuit with marker at 40ns

4 Conclusion

In conclusion, we successfully created and tested a Nand circuits using Verilog and GTK-Wave. Learning how to code in Verilog, at least for our purposes, was relatively simple with few difficulties. Using GTKWave was also fairly intuitive. The most difficulty our group has had so far was most likely creating this report in LaTeX, as we have some trouble with coding certain formatting as well as issues with figures floating around the document. Overall, though, it has been a smooth process and we have all learned quite a bit.